

Considerazioni Aggiuntive Al Testo D El Prof

marco robba

July 19, 2026

Contents

1	Per il DB	1
1.1	Stazione	1
1.1.1	Di tracciamento	1
1.1.2	guasti	2
1.2	Treno	2
2	TAbelle Db	2
2.1	Centrale	2
2.2	Stazione	8
2.3	Treno	9
3	funzionalita da spletare per ogni componente	9
3.1	microservizio treno	9
3.2	microservizio stazione	10
3.3	Centrale	10

1 Per il DB

1.1 Stazione

1.1.1 Di tracciamento

- Quali dati produce la stazione
 1. Treno di passaggio
 - dati dell’ingresso (timestamp e id treno)
 - dati dell’uscita (timestamp e id treno)

sarebbero i sensori della stazione posti alle entrate e alle uscite della stazione

- Eccezione un treno può entrarvi e non uscirvi capolinea
 - * gestire il caso per cui la stazione non è un capolinea ma comunque il treno lo si smonta e lo si ritira da quella stazione, molto spesso a causa di un guasto
- un treno può uscirvi senza entrarvi

1.1.2 guasti

- Stato del guasto(risolto/non risolto)
 1. Gestione guasto nel canale informativo Per non perdere i dati previo descritti allora è cosa buona immagazzinare tali per non perderli e reinviarli una volta che la connessione viene ristabilita e cancellarli una volta che saranno inviati tutti
 2. Guasti vero e proprio della stazione ne inibisce l'uso
 - Esempi
 - * rottura di un binario
 - per una particolare linea della stazione
 - * rottura di un treno
 - per una particolare linea della stazione
 - treno esploso
 - treno deragliato (non credo sia necessario gestire questi casi)

1.2 Treno

- stato: fermo(per via di un guasto su una linea) o attivo guasto etc per gestire se lasciarlo fermo o farlo muovere
 - le proprie informazioni
 - itinerario

2 TA delle Db

2.1 Centrale

=====

SCHEMA DEL DATABASE CENTRALE

Monitoraggio e Gestione del Traffico Ferroviario

Autore: Marco Robba

1. DATABASE CENTRALE

Il `database` centrale raccoglie tutte le informazioni provenienti dai microservizi (stazioni e treni) e le storicizza. Le tabelle si
↪ dividono
in tabelle operative (stato corrente) e tabelle storiche (dati storicizzati).

1.1 Tabelle Operative

--- Stazione ---

```
CREATE TABLE Stazione (  
    id_stazione          INT PRIMARY KEY,  
    nome                 VARCHAR(100) NOT NULL,  
    tipoCapolineaPartenzaoNormale VARCHAR(50)  
        CHECK (tipoCapolineaPartenzaoNormale  
            IN ('capolinea','partenza','normale'))  
);
```

--- Tratte ---

```
CREATE TABLE Tratte (  
    id_Tratta           INT PRIMARY KEY,  
    StazionePartenzaFK  INT NOT NULL,  
    StazioneArrivoFK    INT NOT NULL,  
    FOREIGN KEY (StazionePartenzaFK) REFERENCES  
        ↪ Stazione(id_stazione),  
    FOREIGN KEY (StazioneArrivoFK)  REFERENCES  
        ↪ Stazione(id_stazione)  
);
```

--- Utenti ---

```
CREATE TABLE Utenti (  
    id_utente           INT PRIMARY KEY,  
    nome                 VARCHAR(100) NOT NULL,  
    cognome              VARCHAR(100) NOT NULL,  
    email                VARCHAR(100) NOT NULL,  
    password              VARCHAR(100) NOT NULL,  
    tipo                  VARCHAR(50) NOT NULL,  
    FOREIGN KEY (tipo) REFERENCES  
        ↪ TipoUtente(id_tipo)
```

```

    id_utente INT PRIMARY KEY,
    tipo      VARCHAR(50) NOT NULL,    -- es.
        ↳ 'operatore', 'tecnico', 'admin'
    nome      VARCHAR(100) NOT NULL,
    cognome   VARCHAR(100) NOT NULL,
    matricola VARCHAR(50)  UNIQUE NOT NULL
);

--- Treni ---

CREATE TABLE Treni (
    id_convoglio INT PRIMARY KEY,
    stato        VARCHAR(30) NOT NULL
        CHECK (stato IN ('attivo', 'fermo', 'rotto', 'in
        ↳ manutenzione')),
    itinerario   INT,
    PosizioneAttualeTrattaOStazione INT,
    FOREIGN KEY (itinerario)
        REFERENCES Itinerari(id_itinerario),
    FOREIGN KEY (PosizioneAttualeTrattaOStazione)
        REFERENCES Tratte(id_Tratta)
);

--- Itinerari ---

CREATE TABLE Itinerari (
    id_itinerario INT PRIMARY KEY

);

-- Tabella associativa N:M Itinerari <-> Tratte
-- Un itinerario e' composto da piu' tratte ordinate
CREATE TABLE Itinerario_Tratta (
    id_itinerario INT NOT NULL,
    id_Tratta     INT NOT NULL,
    ordine        INT NOT NULL,    -- posizione della tratta
        ↳ nell'itinerario
    PRIMARY KEY (id_itinerario, id_Tratta),
    FOREIGN KEY (id_itinerario) REFERENCES Itinerari(id_itinerario),
    FOREIGN KEY (id_Tratta)     REFERENCES Tratte(id_Tratta)
);

--- Transiti ---

-- Relazione (0,N) Stazione -- (0,1) Transiti
-- "e_spettatrice_correntemente"

```

```

CREATE TABLE Transiti (
    id_transito      INT PRIMARY KEY,
    id_stazione      INT NOT NULL,
    id_convoglio     INT NOT NULL,
    id_Tratta        INT,
    tempoEntrata     TIMESTAMP NOT NULL,
    tempoUscita      TIMESTAMP,
    FOREIGN KEY (id_stazione) REFERENCES Stazione(id_stazione),
    FOREIGN KEY (id_convoglio) REFERENCES Treni(id_convoglio),
    FOREIGN KEY (id_Tratta)   REFERENCES Tratte(id_Tratta)
);

```

--- Guasti Pervenuti da Treni o Stazioni ---

```

CREATE TABLE Guasti_Pervenuti_da_treni_o_Staz (
    id_Guasto          INT PRIMARY KEY,
    Stato_RisoltoON0   BOOLEAN NOT NULL DEFAULT FALSE,
    OperatoreCheSeNeStaOccupandoFK INT,
    FOREIGN KEY (OperatoreCheSeNeStaOccupandoFK)
        REFERENCES Utenti(id_utente)
);

```

Relazione: perAChiEAssegnataLaRiparazione

La **foreign key** OperatoreCheSeNeStaOccupandoFK nella tabella Guasti_Pervenuti_da_treni_o_Staz realizza la relazione con Utenti ("per a chi è assegnata la riparazione").

1.2 Tabelle Storiche (Storici)

Le tabelle storiche conservano una copia immutabile di ogni evento significativo. Ogni riga possiede un id proprio e i **timestamp** di riferimento.

--- Storico Transiti ---

```

CREATE TABLE Storico_Transiti (
    id_storico_transito INT PRIMARY KEY AUTO_INCREMENT,
    id_transito         INT NOT NULL,
    id_stazione         INT NOT NULL,
    id_convoglio        INT NOT NULL,
    id_Tratta           INT,
    tempoEntrata        TIMESTAMP NOT NULL,
    tempoUscita         TIMESTAMP,

```

```

    ts_storicizzazione    TIMESTAMP NOT NULL DEFAULT
        ↪ CURRENT_TIMESTAMP,
    FOREIGN KEY (id_stazione) REFERENCES Stazione(id_stazione),
    FOREIGN KEY (id_convoglio) REFERENCES Treni(id_convoglio),
    FOREIGN KEY (id_Tratta)    REFERENCES Tratte(id_Tratta)
);

--- Storico Guasti ---

CREATE TABLE Storico_Guasti (
    id_storico_guasto      INT PRIMARY KEY AUTO_INCREMENT,
    id_Guasto              INT NOT NULL,
    Stato_RisoltoONNO      BOOLEAN NOT NULL,
    OperatoreCheSeNeStaOccupandoFK INT,
    ts_apertura            TIMESTAMP NOT NULL,
    ts_chiusura            TIMESTAMP,
    ts_storicizzazione     TIMESTAMP NOT NULL DEFAULT
        ↪ CURRENT_TIMESTAMP,
    FOREIGN KEY (id_Guasto)
        REFERENCES Guasti_Pervenuti_da_treni_o_Staz(id_Guasto),
    FOREIGN KEY (OperatoreCheSeNeStaOccupandoFK)
        REFERENCES Utenti(id_utente)
);

--- Storico Stato Treni ---

CREATE TABLE Storico_Stato_Treni (
    id_storico_treno      INT PRIMARY KEY AUTO_INCREMENT,
    id_convoglio          INT NOT NULL,
    stato                 VARCHAR(30) NOT NULL,
    itinerario            INT,
    PosizioneAttualeTrattaOStazione INT,
    ts_storicizzazione    TIMESTAMP NOT NULL DEFAULT
        ↪ CURRENT_TIMESTAMP,
    FOREIGN KEY (id_convoglio) REFERENCES Treni(id_convoglio)
);

--- Storico Stato Stazioni ---

CREATE TABLE Storico_Stato_Stazioni (
    id_storico_stazione INT PRIMARY KEY AUTO_INCREMENT,
    id_stazione         INT NOT NULL,
    nome                VARCHAR(100) NOT NULL,
    tipoCapolineaPartenzaoNormale VARCHAR(50),
    funzionanteONo      BOOLEAN,

```

```

        ts_storicizzazione    TIMESTAMP NOT NULL DEFAULT
        ↪ CURRENT_TIMESTAMP,
        FOREIGN KEY (id_stazione) REFERENCES Stazione(id_stazione)
    );

    --- Storico Itinerari ---

    CREATE TABLE Storico_Itinerari (
        id_storico_itinerario INT PRIMARY KEY AUTO_INCREMENT,
        id_itinerario         INT NOT NULL,
        id_convoglio          INT NOT NULL,
        ts_assegnazione       TIMESTAMP NOT NULL,
        ts_completamento     TIMESTAMP,
        ts_storicizzazione    TIMESTAMP NOT NULL DEFAULT
        ↪ CURRENT_TIMESTAMP,
        FOREIGN KEY (id_itinerario) REFERENCES Itinerari(id_itinerario),
        FOREIGN KEY (id_convoglio) REFERENCES Treni(id_convoglio)
    );

    --- Storico Assegnazione Operatori a Guasti ---

    CREATE TABLE Storico_Assegnazioni_Guasti (
        id_storico_assegnazione INT PRIMARY KEY AUTO_INCREMENT,
        id_Guasto               INT NOT NULL,
        id_utente               INT NOT NULL,
        ts_assegnazione         TIMESTAMP NOT NULL,
        ts_risoluzione          TIMESTAMP,
        ts_storicizzazione     TIMESTAMP NOT NULL DEFAULT
        ↪ CURRENT_TIMESTAMP,
        FOREIGN KEY (id_Guasto) REFERENCES
            Guasti_Pervenuti_da_treni_o_Staz(id_Guasto),
        FOREIGN KEY (id_utente) REFERENCES Utenti(id_utente)
    );

```

4. RIEPILOGO RELAZIONI DEL DATABASE CENTRALE

Relazione (ER)	Da	A
↪ Cardinalità		

↪ -----		
StazionePartenzaFK	Tratte	Stazione
↪ N:1		
StazioneArrivoFK	Tratte	Stazione
↪ N:1		

e_spettatrice_correntemente	Stazione	Transiti
↪ (0,N) -- (0,1)		
inTransito	Treni	Tratte
↪ N:1		
itinerario	Treni	Itinerari
↪ N:1		
Itinerario_Tratta	Itinerari	Tratte
↪ N:M		
perAChiEAssegnataLaRiparazione	Guasti_Pervenuti	Utenti
↪ N:1		

2.2 Stazione

```

CREATE TABLE Stato (
    id_stazione          INT PRIMARY KEY,
    nomeStazione         VARCHAR(100) NOT NULL,
    regioneGeograficaDiAppartenenza VARCHAR(100),
    funzionanteONo       BOOLEAN,
    flagFaultInCorso     BOOLEAN,
    timestampInizioUltimoFault    TIMESTAMP,
    timestampFineUltimoFault      TIMESTAMP
);

CREATE TABLE Tratte (
    id_tratta          INT PRIMARY KEY, -- deve essere lo stesso del
    ↪ DB centrale
    stazionePartenza  INT NOT NULL,
    stazioneArrivo    INT NOT NULL,
);

CREATE TABLE Guasto_Locale_Momentaneo (
    id_Guasto          INT PRIMARY KEY,
    statoRisoltoONo    BOOLEAN,
    tipo               VARCHAR(50),
    descrizione        VARCHAR(255),
    timestamp          TIMESTAMP
);

-- Tabella associativa per la relazione (0,N)-(0,N) Tratte <->
↪ Guasto_Locale_Momentaneo
CREATE TABLE Tratta_Guasto (
    id_tratta INT NOT NULL,
    id_Guasto INT NOT NULL,
    PRIMARY KEY (id_tratta, id_Guasto),
    FOREIGN KEY (id_tratta) REFERENCES Tratte(id_tratta),

```

```

FOREIGN KEY (id_Guasto) REFERENCES
    ↳ Guasto_Locale_Momentaneo(id_Guasto)
);

CREATE TABLE Treno (
    id_Treno      INT PRIMARY KEY,
    id_transito    INT,
    tempoEntrata   TIMESTAMP,
    tempoPartenza  TIMESTAMP
);

CREATE TABLE bufferChache (
    id_Evento INT PRIMARY KEY,
    tipo      VARCHAR(50),
    id_Guasto INT NULL,
    id_Treno  INT NOT NULL,
    FOREIGN KEY (id_Guasto) REFERENCES
        ↳ Guasto_Locale_Momentaneo(id_Guasto),
    FOREIGN KEY (id_Treno) REFERENCES Treno(id_Treno)
);

```

2.3 Treno

```

CREATE TABLE Stato_convoglio (
    -- Chiave primaria (pallino nero)
    id_convoglio INT PRIMARY KEY,

    -- Attributi (pallino bianco)
    direzione VARCHAR(50) NOT NULL,
    stazione_corrente VARCHAR(100),

    -- Gestione dello stato con vincolo di controllo (o ENUM in
    ↳ MySQL)
    stato VARCHAR(30) NOT NULL CHECK (stato IN ('attivo', 'fermo',
    ↳ 'rotto', 'in manutenzione')),

    prossima_stazione VARCHAR(100),
    tratta_corrente VARCHAR(100)
);

```

3 funzionalita da spletare per ogni componente

3.1 microservizio treno

- aggiornamento tratta

- salvarla in ram per il proprio uso e sul db
- gestione causa guasti alla prossima fermata stazione o binario (trattata ovvero copia di stazioni)
 - ricezione dai sensori → modifica della percorrenza della tratta, sincronizzazione sui topic di allarme delle nuove stazioni
 - * se arrivano eventi del tipo la prossima stazione è fuori uso aggiornare variabile in ram e il database che il treno si è fermato e
 - * se l'evento è il ripristino della funzionalità della stazione allora riaggiornare la ram e il db che il treno è partito
- Ritardi la tratta possiede orari corretti di percorrenza
- inviare in centrale l'entrata e uscita da una stazione topic treni/+ /passaggio formato : boh lo fa l'ai
- invio se il treno si guasta alla centrale

3.2 microservizio stazione

- invio alla centrale i dati del passaggio dei treni
 - i sensori della stazione informano id treno e se è in entrata o in uscita, poi si aggiorna la ram e si inviano i dati alla stazione centrale per la storicizzazione
- la stazione può perdere la connessione con la centrale nel caso seguire il diagramma degli stati che ne descrive la dinamica di gestione

3.3 Centrale

- deve rispondere alle richieste di visualizzazione
 - montare su il motore in tempo reale, cicli continui che inviano i dati se la richiesta è ancora attiva
- formattare e storicizzare i dati

L'unica crud a cui serve la propagazione ai microservizi è la

- gestione guasti
 - se i microservizi avvertono guasti , aggiornamento del db,
 - se il tecnico invia l'operatore aggiornare il db di modo che ormai il guasto sia terminato e pubblicare sul topic degli alert generale per tutti i treni o tutte le stazioni che quel tale microservizio è stato risolto

interrogare il database ogni 10 secondi per ogni tipo di visualizzazione di modo da rendere il “tempo reale”